



HW 3

Default Code

Below is the default settings for the code provided by the website

https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html#sphx-glr-beginner-blitz-cifar10-tutorial-py

```
In [ ]: import torch
import torchvision
import torchvision.transforms as transforms
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 4
#Download data
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                           shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                          shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
```

```

        return x

net = Net()
import torch.optim as optim
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
for epoch in range(2): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999: # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
            running_loss = 0.0

print('Finished Training')
correct = 0
total = 0
# since we're not training, we don't need to calculate the gradients for our c
with torch.no_grad():
    for data in testloader:
        images, labels = data
        # calculate outputs by running images through the network
        outputs = net(images)
        # the class with the highest energy is what we choose as prediction
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct // to
correct = 0
total = 0
# since we're not training, we don't need to calculate the gradients for our c
with torch.no_grad():
    for data in trainloader:
        images, labels = data
        # calculate outputs by running images through the network
        outputs = net(images)
        # the class with the highest energy is what we choose as prediction

```

```
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the training images: {100 * correct // total}')
```

For our default settings, the training and testing accuracy was 58% and 55% respectively.

0.1

Channels

First, we will modify the number of channels in each layer by doubling them. The important parts we changed in our default settings are as follows.

```
In [ ]: class Net(nn.Module):
        def __init__(self):
            super().__init__()
            self.conv1 = nn.Conv2d(3, 12, 5) #Double # of channels from 6 to 12
            self.pool = nn.MaxPool2d(2, 2)
            self.conv2 = nn.Conv2d(12, 32, 5) #and 16 to 32
            self.fc1 = nn.Linear(32 * 5 * 5, 240)
            self.fc2 = nn.Linear(240, 168)
            self.fc3 = nn.Linear(168, 10)
        def forward(self, x):
            x = self.pool(F.relu(self.conv1(x)))
            x = self.pool(F.relu(self.conv2(x)))
            x = torch.flatten(x, 1) # flatten all dimensions except batch
            x = F.relu(self.fc1(x))
            x = F.relu(self.fc2(x))
            x = self.fc3(x)
            return x
```

Doubling the # of channels resulted in the accuracy improving by 6% for both the training and test dataset at 64% and 61% respectively.

No. Convolutional Layers

For the next part, we will adjust the number of convolutional layers to 1 and 3. The parts of the code we changed are as follows.

```
In [ ]: # For 1 convolutional layer
        class Net(nn.Module):
            def __init__(self):
                super().__init__()
```

```

        self.conv1 = nn.Conv2d(3, 6, 5) #Reduce to 1 convolutional layer
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(6*14*14, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
#For 3 convolutional layers

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.conv3 = nn.Conv2d(16, 32, 3) #Add 3rd convolutional layer
        self.pool = nn.MaxPool2d(2, 2)

        # Output from conv3 is (32 * 1 * 1)
        self.fc1 = nn.Linear(32 * 1 * 1, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

```

Compared to the default of 2 layers, reducing the number of convolutional layers to 1 didn't affect the training and test accuracy much at all and stayed relatively the same at 58% and 54% but increasing the number of convolutional layers to 3 dramatically worsened the training and test accuracy to both 45%.

Activation Functions

In our default settings, we already use ReLU so let's see what happens when we change the activation function to leaky ReLU and tanh. The parts of the code we modified are

```

In [ ]: # For leaky ReLU
class Net(nn.Module):

```

```

def __init__(self):
    super().__init__()
    self.conv1 = nn.Conv2d(3, 6, 5)
    self.pool = nn.MaxPool2d(2, 2)
    self.conv2 = nn.Conv2d(6, 16, 5)
    self.fc1 = nn.Linear(16 * 5 * 5, 120)
    self.fc2 = nn.Linear(120, 84)
    self.fc3 = nn.Linear(84, 10)

def forward(self, x):
    x = self.pool(F.leaky_relu(self.conv1(x)))
    x = self.pool(F.leaky_relu(self.conv2(x)))
    x = torch.flatten(x, 1) # flatten all dimensions except batch
    x = F.leaky_relu(self.fc1(x))
    x = F.leaky_relu(self.fc2(x))
    x = self.fc3(x)
    return x

# For tanh
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.tanh(self.conv1(x)))
        x = self.pool(F.tanh(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.tanh(self.fc1(x))
        x = F.tanh(self.fc2(x))
        x = self.fc3(x)
        return x

```

When we used ReLU in our default settings the training and test accuracy was 58% and 55% respectively. When tanh and leaky ReLU was used, the performance did not change much. Leaky ReLU did slightly better with training and test accuracies 58% and 56% while tanh did slightly worse at 55% and 53%.

0.2

Learning Rate

Let's see what happens when we increase and decrease the learning rate. The parts we changed are as follows

```
In [ ]: #Bigger learning rate
optimizer = optim.SGD(net.parameters(), lr=0.1, momentum=0.9)

#Smaller learning rate
optimizer = optim.SGD(net.parameters(), lr=0.00001, momentum=0.9)
```

When we either increased or decreased the learning rate, the performance was significantly worse. If we increase the learning rate the training and test accuracy were both 10% while decreasing the learning rate the training and test accuracy was both 9%.

Optimizer

Next, let's see what happens when we change the optimizer to SGD without momentum and to Adam. The modified code is as follows.

```
In [ ]: # SGD w/out momentum
optimizer = optim.SGD(net.parameters(), lr=0.001)

#Adam
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

Compared to the default, if we use the optimizer SGD w/ out momentum, the model performs much worse with training and test accuracies at 33% and 34% respectively. If we use the optimizer Adam, the model performs slightly better with training and test accuracies 61% and 58%.

Batch Size

We will now observe what will happen if we increase the batch size from 4 to 32. The parts we changed are

[illegible]

If we increase the batch size to 32, compared to the default, the model performs noticeably worse with training and test accuracies of 38%.

Longer Training Time

Finally let's see what happens if we increase the number of epochs from 2 to 5. The parts of the code we changed are

```
In [ ]: for epoch in range(5): # change # of epochs from 2 to 5

        running_loss = 0.0
        for i, data in enumerate(trainloader, 0):
            # get the inputs; data is a list of [inputs, labels]
            inputs, labels = data

            # zero the parameter gradients
            optimizer.zero_grad()

            # forward + backward + optimize
            outputs = net(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            # print statistics
            running_loss += loss.item()
            if i % 2000 == 1999: # print every 2000 mini-batches
                print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
                running_loss = 0.0
```

The results show that if we increase the number of epochs from 2 to 5, the model performs slightly better with training and test accuracies of 66% and 61%.

```
In [ ]:
```